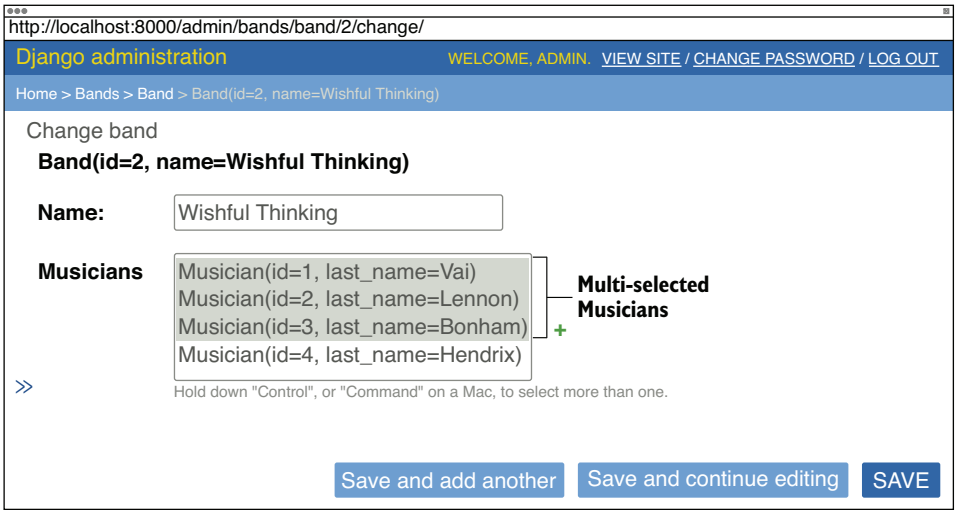


Figure 5.10 `Band` **Change screen with associated** `Musician` **objects**

For large datasets this screen can be rather slow to load. To populate the Musician Multiselect, every possible `Musician` object needs to be queried and shown. One way to improve the performance is to keep the `.__str__()` result as short and simple as possible.

You saw how the Django Admin uses query parameters to display a subset of the results on the listing page when you use filters. Whether or not you have a filter, you can take advantage of this feature in your own code. You can add query parameters to the URL and change what is shown on the screen. This is handy if you want to display a single object without using the Change page. For example, http://localhost:8000/admin/bands/band/?id=2 displays the listing page with only the `Band` with an ID of 2 in the list.

The filter query parameters also support the same field lookup modifiers as the ORM. The http://localhost:8000/admin/bands/musician/?id__gte=3 URL shows the Musician Listing page containing only those `Musician` objects with an ID greater than or equal to 3. A more useful field lookup modifier is `__in`; passing it a comma-separated list of numbers limits the content of a listing page to just those IDs.

Using this feature, you can add a column to the listing page with clickable links to related objects—for example, a column on the Musician page that links to the musician's bands. To do this, add another callable that returns a link. You don't want to hard-code the link to the Band Listing page; you want to look it up. Recall the `{% url %}` that does this kind of lookup in a template—the Python code equivalent is the `reverse()` function in the `django.urls` module. All the Django Admin URLs are named, so if you know the pattern, you can look up other Django Admin pages using `reverse()`.

The name of a Django Admin listing page is `admin:`*`module_class`*`_changelist`, where *module* and *class* get replaced with the module and class name of the `Model`